Unit 3

Data Understanding

- **Data understanding** basically involves **analyzing and exploring the data to identify any patterns or trends t**hat may be present.

- The data understanding phase typically involves the following steps −

- **Data Collection** −
  - This involves gathering the relevant data that you will be using for your analysis.
  - The data can be collected from various sources such as databases, websites, and APIs.

- **Data Cleaning** −
  - This involves cleaning the data by removing any irrelevant or duplicate data, and dealing with missing data values.
  - The data should be formatted in a way that makes it easy to analyze.

Data Understanding

- **Data Exploration** –
  - This involves exploring the data to identify any patterns or trends that may be present.
  - This can be done using various statistical techniques such as histograms, scatter plots, and correlation analysis.

- **Data Visualization** –
  - This involves creating visual representations of the data to help you understand it better.
  - This can be done using tools such as graphs, charts, and maps.

- **Data Preprocessing** –
  - This involves transforming the data to make it suitable for use in machine learning algorithms.
  - This can include scaling the data, transforming it into a different format, or reducing its dimensionality.

Data Preparation

https://www.tutorialspoint.com/machine_learning/machine_learning_preparing_data.htm

- **Data preparation** is a critical step in the machine learning process, and can have a significant **impact on the accuracy and effectiveness of the final model**.
- It requires careful attention to detail and a thorough understanding of the **data and the problem** at hand**.**

- **What is Data Preparation?**
- Data preparation is the **process of dealing with raw data** i.e, cleaning, organizing and transforming it to align with the **machine learning algorithms.**

- Data preparation is a **continuous process,** and has a huge impact on the performance of machine learning model.

- **Clean and structured data** would result in better outcomes.

Data Preparation

Importance of Data Preparation:

- **Improves model accuracy** –
  - **Machine learning algorithms** rely completely on data.
  - When you **provide clean and structured data to models**, the outcomes are accurate.

- **Facilitates Feature Engineering** –
  - **Data preparation** often includes the process of selecting or creating new features to train the model.
  - Hence, **data preparation would make feature engineering easy.**

- **Data Quality** −
  - **Collected data** most often would contain inconsistencies, errors and irrelevant information.
  - Hence when tasks **like data cleaning, transformation are applied**, the data is **formatted and neat.** This can be used for **gaining insights and patterns.**

- **Enables rate of prediction** −
  - **Prepared data** makes it easier to analyze results and would yield **accurate outcomes.**

Data Preprocessing

https://www.geeksforgeeks.org/machine-learning/data-preprocessing-machine-learning-python/

- **Data preprocessing** is the first step in any data analysis or machine learning pipeline.
- It **involves** cleaning, transforming and organizing raw data to ensure it is **accurate, consistent and ready for modeling.**

- It has a big impact on model building such as:
- **Clean and well-structured data allows models** to learn meaningful patterns rather than noise.

- **Properly processed data prevents** misleading inputs, leading to more reliable predictions.

- **Organized data makes it simpler** to create useful inputs for the model, enhancing model performance.

- **Organized data supports** better Exploratory Data Analysis (EDA), making patterns and trends more interpretable.

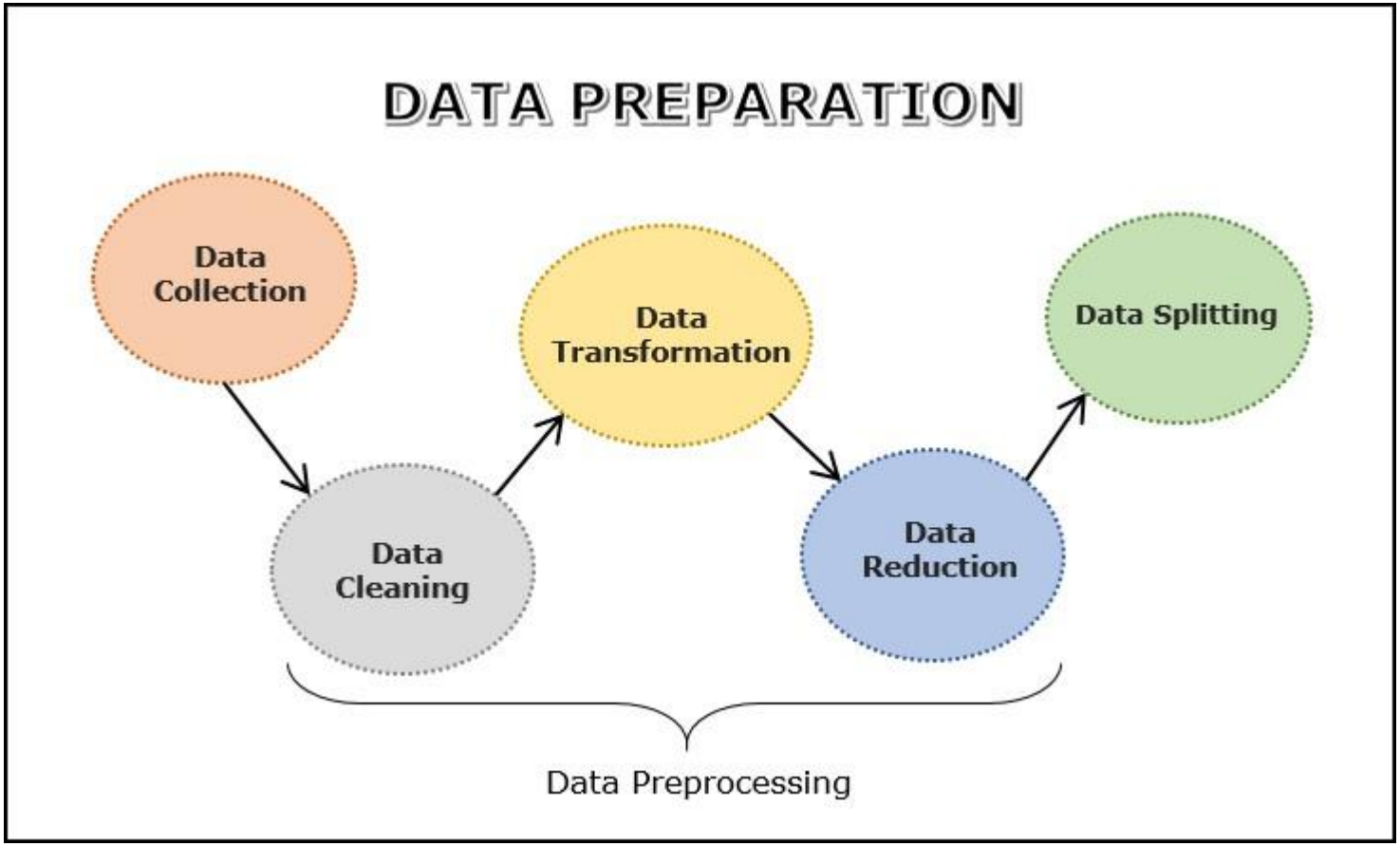
- **Advantages**
- Let's see the advantages of data preprocessing,
- **Improves Data Quality:** Cleans and organizes raw data for **better analysis.**
- **Enhances Model Accuracy:** Removes noise and irrelevant data, leading to **more precise predictions.**
- **Reduces Overfitting:** Handles outliers and redundant features, improving **model generalization.**
- **Speeds Up Training:** Efficiently scaled data **reduces computation time.**
- **Ensures Algorithm Compatibility:** Converts data into formats suitable for machine learning models

Data Preparation Process Steps

- **Data preparation process** involves a **sequence of steps** that is required to make data suitable for analysis and modeling.

- **The goal of data preparation** is to make sure that the data is accurate, complete, and relevant for the analysis.

- The following are some of the key steps involved in data preparation −
  - Data Collection
  - Data Cleaning
  - Data Transformation
  - Data Reduction
  - Data Splitting


DATA PREPARATION
Data Collection
Data Cleaning
Data Transformation
Data Reduction
Data Splitting
Data Preprocessing

Data Collection

- **Data collection** is the first step in the process of machine learning, where
  - data from different sources is gathered to make decisions,
  - answer research questions and
  - statistical planning.

- Different sources such as **databases, text files, pictures, sound files, or web scraping** may be used for data collection.

- Once the data is selected, the data has to be preprocessed in order to gain insights.

- This process is carried out **to put the data in an appropriate format** that would be useful for problem solving.

- **Some time data collection** follows the data integration step.

- **Data integration** involves combining data from multiple sources into a **single dataset for analysis**.

- **This may involve** matching or linking records across different datasets, **or merging datasets based on common variables.**

- **After selecting the raw data**, the most important task is **data preprocessing**.

- In broad sense, **data preprocessing** will **convert the selected data into a form** we can work with or can feed to ML algorithms.

- The data preprocessing includes
  - **data cleaning, transformation and reduction.**

Data Cleaning

- **Data cleaning** is the process of identifying and correcting errors, missing values, duplicate values and outliers, etc. in the data.

- This step is crucial in the process of machine learning as it ensures that the data is accurate, relevant and error free.

- **Common techniques used** for data cleaning include imputation, outlier detection and removal, etc.

- The following is a sequence of steps for data cleaning −
  - 1. Handling duplicate values
  - 2. Fixing syntax errors
  - 3. Dealing outliers
  - 4. Handling Missing Values
  - 5. Validating the data

-

- **1. Handling duplicate values**
- Duplicates in the dataset means that **there is repeated data**, which might occur due to data entry errors or issues while collecting data.

- The **technique used to remove duplicates** is first they are identified and then deleted using **drop_duplicates function in Pandas**.

- **2. Fixing syntax errors**
- In this step, structural errors like inconsistencies in data format or naming conventions should be addressed.

- Standardizing formats and fixing errors would **ensure data consistence and accurate analysis.**

- **3. Dealing outliers**
- Outliers are values that are unusual and differ greatly with the data.

- **The techniques** used to **detect outliers include statistical methods** like **z-score or IQR (**Interquartile Range**) method and machine learning methods like clustering and SVM's (Support Vector Machine).**

- **4. Handling Missing Values**
- Missing values are the **values or data that is not stored for some values in the dataset.**

- There are several ways to handle missing data like:
- **Imputation** –
  - In this process the **missing values are substituted with different value**, which can be a central tendency measure like **mean, median or mode for numeric** values and most **frequency category for categorical data.**
  - Some other methods in imputation include regression imputation and multiple imputation.

- **Deletion** –
  - In this process the entire instances with missing values are removed.
  - Well, this is **not a reliable method since there is loss of data.**

- **5. Validating the data**
- **Data Validation** is another stage that makes sure that the data aligns perfectly with the requirements so that the **predicted outcome is accurate.**

- Some common data validation procedures it the correctness of data before storing them in databases are:
  - Data type check
  - Code Check
  - Format check
  - Range check

Advantages

https://www.geeksforgeeks.org/data-analysis/data-cleansing-introduction/

- **Improved model performance:** Removal of errors, inconsistencies and irrelevant data helps the model to better **learn** from the data.

- **Increased accuracy:** Helps ensure that the data is **accurate, consistent and free of errors.**

- **Better representation of the data:** Data cleaning allows the **data to be transformed into a format** that better represents the **underlying relationships and patterns in the data.**

- **Improved data quality:** Improve the **quality of the data**, making it more reliable and accurate.

- **Improved data security:** Helps to **identify and remove sensitive or confidential information t**hat could compromise data security.

Data Transformation

- **Data transformation** is the **process** of converting the data from its original format into a format that is suitable for **analysis and modeling**.

- This could include defining the structure, aligning the data, extracting data from source, and then storing it in an appropriate form.

- There are many techniques **available to transform data into a suitable format**.

- Some commonly used **data transformation techniques** are as follows −
  - Scaling
  - Normalization − L1 & L2 Normalizations
  - Standardization
  - Binarization
  - Encoding
  - Log Transformation

1.Scaling:

- In most cases, the **data we collected consists of attributes** with **varying scale**, but we **cannot provide such data** to **ML** algorithm hence it requires rescaling.

- **Data scaling** makes sure that attributes are at the same scale i.e, **usually range of 0 to 1.**

- We can **rescale** the data with the help of **MinMaxScaler** class of scikit-learn Python library.

- https://www.tutorialspoint.com/machine_learning/machine_learning_preparing_data.htm (precision=1)

- https://www.kaggle.com/code/sugamkhetrapal/project-4-pima-indians-diabetes
- (precision=3)

Pregnanci	Glucose	BloodPres	SkinThickr	Insulin	BMI	DiabetesF	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
5	116	74	0	0	25.6	0.201	30	0
3	78	50	32	88	31	0.248	26	1
10	115	0	0	0	35.3	0.134	29	0
2	197	70	45	543	30.5	0.158	53	1
8	125	96	0	0	0	0.232	54	1
4	110	92	0	0	37.6	0.191	30	0

- Decribe()

	preg	plas	pres	skin	test	mass	pedi	age	class
count	768.000	768.000	768.000	768.000	768.000	768.000	768.000	768.000	768.000
mean	3.845	120.895	69.105	20.536	79.799	31.993	0.472	33.241	0.349
std	3.370	31.973	19.356	15.952	115.244	7.884	0.331	11.760	0.477
min	0.000	0.000	0.000	0.000	0.000	0.000	0.078	21.000	0.000
25%	1.000	99.000	62.000	0.000	0.000	27.300	0.244	24.000	0.000
50%	3.000	117.000	72.000	23.000	30.500	32.000	0.372	29.000	0.000
75%	6.000	140.250	80.000	32.000	127.250	36.600	0.626	41.000	1.000
max	17.000	199.000	122.000	99.000	846.000	67.100	2.420	81.000	1.000

https://www.kaggle.com/code/sugamkhetrapal/project-4-pima-indians-diabetes/input

https://www.kaggle.com/code/sugamkhetrapal/project-4-pima-indians-diabetes

- X_new = (X - X_min)/(X_max - X_min)

- For feature column- Pregnancy

min	0
mox	17

Pregnanci	Glucose	BloodPres	SkinThickr	Insulin	BMI	DiabetesF	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
5	116	74	0	0	25.6	0.201	30	0
3	78	50	32	88	31	0.248	26	1
10	115	0	0	0	35.3	0.134	29	0
2	197	70	45	543	30.5	0.158	53	1
8	125	96	0	0	0	0.232	54	1
4	110	92	0	0	37.6	0.191	30	0

```
(precision=3)
```

```
(precision=1)
```

```
[[0.353 0.744 0.59  0.354 0.    0.501 0.234 0.483]
 [0.059 0.427 0.541 0.293 0.    0.396 0.117 0.167]
 [0.471 0.92  0.525 0.    0.    0.347 0.254 0.183]
 [0.059 0.447 0.541 0.232 0.111 0.419 0.038 0.   ]
 [0.    0.688 0.328 0.354 0.199 0.642 0.944 0.2  ]]
```

```
Scaled data:
[
   [0.4 0.7 0.6 0.4 0.  0.5 0.2 0.5 1. ]
   [0.1 0.4 0.5 0.3 0.  0.4 0.1 0.2 0. ]
   [0.5 0.9 0.5 0.  0.  0.3 0.3 0.2 1. ]
   [0.1 0.4 0.5 0.2 0.1 0.4 0.  0.  0. ]
   [0.  0.7 0.3 0.4 0.2 0.6 0.9 0.2 1. ]
   [0.3 0.6 0.6 0.  0.  0.4 0.1 0.2 0. ]
   [0.2 0.4 0.4 0.3 0.1 0.5 0.1 0.1 1. ]
   [0.6 0.6 0.  0.  0.  0.5 0.  0.1 0. ]
   [0.1 1.  0.6 0.5 0.6 0.5 0.  0.5 1. ]
   [0.5 0.6 0.8 0.  0.  0.  0.1 0.6 1. ]
]
```

https://www.mygreatlearning.com/blog/data-normalization-and-standardization/#:~:text=to%20misleading%20results.-,Conclusion,significant%20benefits%20when%20applied%20appropriately.

2. Normalization:

- Normalization is used to rescale the data with a **distribution value between 0 and 1**.
- For every feature, the minimum value is set to 0 and the **maximum value is set to 1.**
- This is used to rescale each row of data to have a length of 1.
- It is mainly useful in **Sparse dataset** where we **have lots of zeros.**
- We can **rescale the data** with the help of **Normalizer class** of **scikit-learn** Python library.
- In machine learning, there are **two types** of normalization preprocessing techniques as follows −
  - L1 Normalization
  - L2 Normalization

https://www.mygreatlearning.com/blog/data-normalization-and-standardization/#:~:text=to%20misleading%20results.-,Conclusion,significant%20benefits%20when%20applied%20appropriately.

- L1 Normalization
- It may be defined as **the normalization technique** that modifies the dataset values in a way that in each row the sum of the absolute values will always be up to 1.
- It is also called **Least Absolute Deviations.**

- L2 Normalization
- It may be defined as **the normalization technique** that modifies the dataset values in a way that in each row the sum of the squares will always be up to 1.
- It is also **called least squares.**

- For additional -- https://www.mygreatlearning.com/blog/data-normalization-and-standardization/#:~:text=to%20misleading%20results.-,Conclusion,significant%20benefits%20when%20applied%20appropriately.\

3. Standardization:

- **Standardization** is used to transform data attributes to a standard Gaussian distribution with a **mean of 0** and a **standard deviation of 1.**

- This technique is **useful in ML** algorithms like **linear regression, logistic regression** that assumes a Gaussian distribution in input dataset and produce better results with rescaled data.

$$x_{i(norm)} = \frac{x_i}{||x||_2}:$$

- We can standardize the data (mean = 0 and SD =1) with the help of StandardScaler class of **scikit-learn** Python library.

https://www.geeksforgeeks.org/machine-learning/normalization-vs-standardization/

S. No.	Normalization	Standardization
1.	Minimum and maximum value of features are used for scaling	Mean and standard deviation is used for scaling.
2.	It is used when features are of different scales.	It is used when we want to ensure zero mean and unit standard deviation.
3.	Scales values between [0, 1] or [-1, 1].	It is not bounded to a certain range.
4.	It is really affected by outliers.	It is much less affected by outliers.
5.	Scikit-Learn provides a transformer called MinMaxScaler for Normalization.	Scikit-Learn provides a transformer called StandardScaler for standardization.
6.	This transformation squishes the n-dimensional data into an n-dimensional unit hypercube.	It translates the data to the mean vector of original data to the origin and squishes or expands.
7.	It is useful when we don't know about the distribution	It is useful when the feature distribution is Normal or Gaussian.
8.	It is a often called as Scaling Normalization	It is a often called as Z-Score Normalization.

4. Binarization :

- As the name suggests, this is the **technique with the help of** which we can make our **data binary**.

- We can use a binary threshold for making our data binary.
- The values **above that threshold** value will be converted to 1 and **below that threshold** will be converted to 0.

- For example, if we **choose threshold value = 0.5**, then the dataset value above it will become 1 and below **this will become 0.** That is why we can call it **binarizing** the data or **thresholding** the data.

- This technique is useful when we have **probabilities in our dataset** and want to convert them into crisp values. (In machine learning, **crisp values** refer to precise, exact, and non-fuzzy data points or classifications (e.g., a exact temperature of 25∘or a binary 0/1 classification))
- We **can binarize the data** with the help of Binarizer class of scikit-learn Python library.

5. Encoding :

- This technique is used to convert categorical variables into numerical representations.
- Some common encoding techniques include **one-hot encoding**, **label encoding** and **target encoding**.

1) Label Encoding:

- Most of the sklearn functions expect that the data with number labels rather than word labels.
- Hence, we need to convert such labels into number labels. This process is called label encoding.
- We can perform label encoding of data with the help of **LabelEncoder() function of scikit-learn** Python library**.**

Example

Sample Data:

cars = ["Manual", "Automatic", "Manual", "CVT", "Automatic"]

- **The Mapping:**
  - **0:** Automatic (comes first alphabetically)
  - **1:** CVT
  - **2:** Manual

- **The Output:** [2, 0, 2, 1, 0]

- **Inverse Transform:** If you need to **convert the numbers back to text later**, use **le.inverse_transform([0, 1, 2]),**
- which will return ['Automatic', 'CVT', 'Manual']

6. Log Transformation:

- This technique is usually used in handling **skewed data**.

- Skewed data in machine learning refers to an **uneven or asymmetrical distribution of values** in a dataset, where data points are concentrated more on one side, resulting in a **longer "tail" on the other**. This can cause machine learning models to be biased and perform poorly on the less frequent (minority) values.

- It involves apply natural **logarithmic function** for all values in the dataset to modify **the scale of numeric values.**

Data Reduction

- **Data Reduction is a technique** to reduce the **size of the dataset** by selecting a subset of features or observations that are most relevant for the analysis.

- This can help to reduce noise and **improve the accuracy** of the model.

- This is useful when the dataset is **very large** or when a dataset contains large amount of irrelevant data.

- One of the most common technique used is **Dimensionality Reduction**, which reduces **the size of the dataset** without loosing the important information.

- Other method is the **Discretization**, where **continuous values** like **time** and **temperature** are converted to **discrete categories** which **simplifies the data.**

Data Splitting

- **Data Splitting** is the last step in the **preparation of data** for machine learning, where the data is split into different sets –

- **Training** − **subset which** is used by the **machine learning** model for **learning patterns**.

- **Validation** − **subset** used to evaluate the performance of machine learning model while training.

- **Testing** − **subset** used to evaluate the performance and **efficiency** of the trained model.

Data Visualization

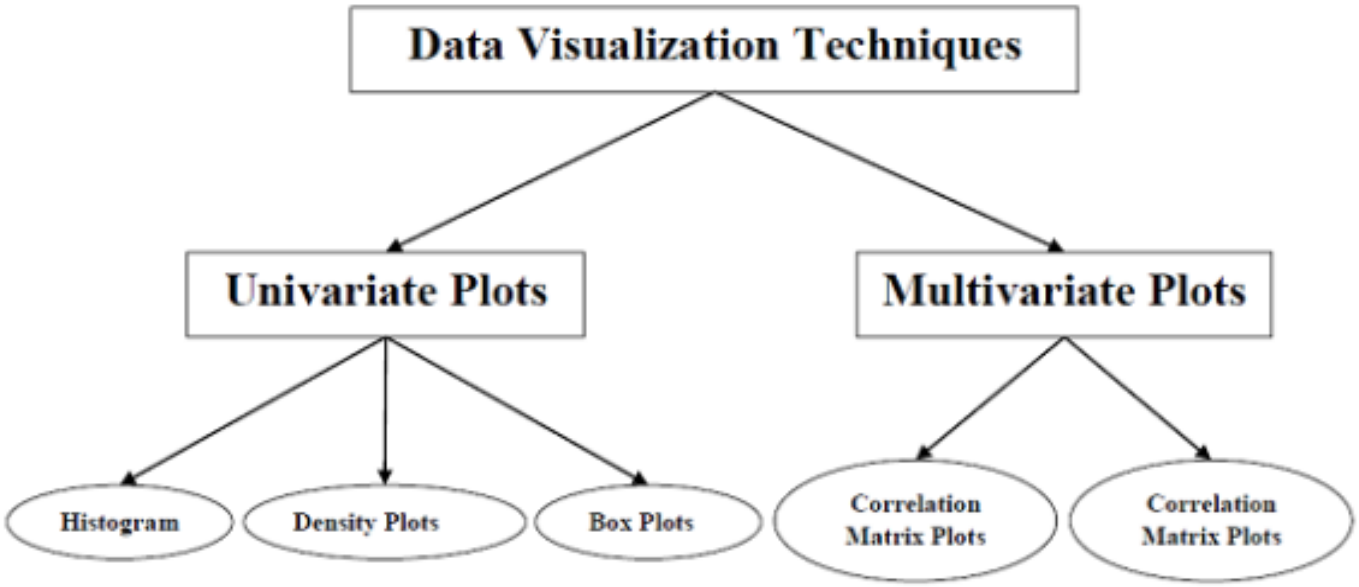
- **Data visualization is** a **graphical representation of data and information.**

- With **the help of data visualization**, we can see **how the data looks like** and **what kind of correlation is held by the attributes of the data.**

- It is the **fastest way to see** if the features **correspond to the output.**

Types of Data Visualization

- Data visualization for machine learning data can be classified into two different categories as follows -
  - Univariate Plots
  - Multivariate Plots


Data Visualization Techniques
Univariate Plots
Multivariate Plots
Histogram
Density Plots
Box Plots
Correlation Matrix Plots
Correlation Matrix Plots

•**Univariate Plots:**

- **Understanding Attributes Independently**

- The simplest type of **visualization is single-variable or univariate visualization**.

- With the help of univariate visualization, we can understand each attribute of our dataset independently.

- The following are some techniques in Python to implement univariate visualization
  - Histograms
  - Density Plots
  - Box and Whisker Plots

Multivariate Plots:

- **Interaction Among Multiple Variables**

- Another type of visualization **is multi-variable or multivariate visualization**.

- With the help of multivariate visualization, we can understand **the interaction between multiple attributes of our dataset.**

- The following are some techniques in Python to implement multivariate visualization −
  - Correlation Matrix Plot
  - Scatter Matrix Plot

Exploratory Data Analysis

- Exploratory Data Analysis (EDA) is an important step in **data science and data analytics** as it visualizes data to understand its main **features, find patterns and discover how different parts of the data are connected.**
-

Why Exploratory Data Analysis Important

- Helps to **understand the dataset by** showing **how many features it has, what type of data each feature contains and how the data is distributed.**

- **Helps to identify hidden patterns and relationships between different data points** which help us in and model building.

- Allows to identify errors or unusual data points (outliers) that could affect our results.

- The insights gained from EDA help us to identify most important features for building models and guide us on how to **prepare them for better performance.**

- By understanding the data it **helps us in choosing best modeling techniques and adjusting them for better results.**

Steps for Performing Exploratory Data Analysis

- It involves a s**eries of steps to help us understand** the data, uncover patterns, identify anomalies, test hypotheses and ensure the data is clean and ready for further analysis.
- It can be done using different tools like:
  - In Python,
    - **Pandas** is used to **clean, filter and manipulate data.**
    - **Matplotlib** helps to create basic visualizations while
    - Seaborn makes more attractive plots.
    - For **interactive visualizations Plotly is a good choice.**
  - In R,
    - **ggplot2** is used for **creating complex plots**,
    - **dplyr** helps with data manipulation and
    - **tidyr** makes sure our data is organized and easy to work with.

Steps for Performing Exploratory Data Analysis

- Its step includes:
  - **Step 1: Understanding the Problem and the Data**
  - **Step 2: Importing and Inspecting the Data**
  - **Step 3: Handling Missing Data**
  - **Step 4: Exploring Data Characteristics**
  - **Step 5: Performing Data Transformation**
  - **Step 6: Visualizing Relationship of Data**
  - **Step 7: Handling Outliers**
  - **Step 8: Communicate Findings and Insights**

Checking dimensions: df.shape

- **Shape Function (df.shape**):
- The **df.shape** function returns **a tuple representing dimensions of the DataFrame.**
- It provides the **number of rows (records) and columns (attributes) in the DataFrame.**
- ***Syntax***: *dataframe.shape*
- **Return** : A tuple in the form of (rows, columns).

Here we use **pandas library to read** and then it prints the shape of the **DataFrame which includes the number of rows and columns.**

```
import pandas as pd

data = pd.read_csv("/content/nba.csv")    // This reads the NBA dataset and stores it
                                          //in a pandas DataFrame called data.

shape = data.shape                        // This attribute returns a tuple representing the
                                          //dimensionality of the DataFrame, formatted
                                          //as (number_of_rows, number_of_columns).

print("Shape = {}".format(shape))
```

Output:

Shape = (458, 9)

Explanation –

The dataset has **458 records** and **9 attributes** for each record.

Listing features: df.columns.

- **Pandas DataFrame.columns:**
- In Pandas, **DataFrame.columns** attribute returns the **column names** of a DataFrame.
- It gives acces**s to the column labels, returning an Index object with the column labels** that may be used for viewing, modifying, or creating new column labels for a DataFrame.

- ***Note:*** *This* ***attribute doesn't require any parameters*** *and simply returns the column labels of the DataFrame when called.*

Retrieve Column Labels Using DataFrame.columns

Here's an example showing how to use **DataFrame.columns** in order to obtain column labels from a DataFrame.

```
import pandas as pd

df = pd.DataFrame({
    'Weight': [45, 88, 56, 15, 71],
    'Name': ['Sam', 'Andrea', 'Alex’,
 'Robin', 'Kia'],
    'Age': [14, 25, 55, 8, 21] })

index = ['Row_1', 'Row_2', 'Row_3', ‘
Row_4', 'Row_5']
df.index = index

print("DataFrame:")
display(df)

result = df.columns

print("\n Name of Columns of Pandas DataFrame")
print(result)
```

DataFrame:

	Weight	Name	Age
Row_1	45	Sam	14
Row_2	88	Andrea	25
Row_3	56	Alex	55
Row_4	15	Robin	8
Row_5	71	Kia	21

```
 Name of Columns of Pandas DataFrame
Index(['Weight', 'Name', 'Age'], dtype='object')
```

Data Selection & Slicing: Selecting specific columns and rows using loc[] and iloc[].

- **Selecting Rows and Columns Using .loc[] (Label-Based Indexing)**

- The **.loc[] method** selects **data based on labels (names of rows or columns).**
- It is **flexible and supports various operations like**
  - selecting single rows/columns,
  - multiple rows/columns, or
  - specific subsets.

- **Key Features of .loc[]:**
  - Label-based indexing.
  - Can select both rows and columns simultaneously.
  - Supports slicing and filtering.

Select a Single Row by Label:

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}

df = pd.DataFrame(data)
print(df,"\n")

row = df.loc[0] # Select the first row
print(row,"\n")
```

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

Name        Alice
Age            25
City     New York
Name: 0, dtype: object
```

- **Output:**
- Name Alice
  Age 25
  City New York
  Name: 0, dtype: object

- When you select a single row using df.loc[0], Pandas converts that row into a **Series**. The extra information at the bottom is "metadata" that describes what that Series is.

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']}

df = pd.DataFrame(data)
print(df,"\n")

row = df.loc[0]
print(row,"\n")

row = df.loc[[0]]
print(row)
```

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

Name       Alice
Age           25
City    New York
Name: 0, dtype: object

    Name  Age      City
0  Alice   25  New York
```

- **Select Multiple Rows by Labels:**

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}

df = pd.DataFrame(data)
print(df,"\n")

rows = df.loc[[0, 2]]  # Select rows with index labels 0 and 2
print(rows)
```

Output:

```
      Name  Age      City
0    Alice   25  New York
2  Charlie   35   Chicago
```

Output

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

      Name  Age      City
0    Alice   25  New York
2  Charlie   35   Chicago
```

Select Specific Rows and Columns:

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}

df = pd.DataFrame(data)
print(df,"\n")
```

```
subset = df.loc[0:1, ['Name', 'City']]  # Select first two rows and specific columns
print(subset)
```

Output:

```
   Name         City
0 Alice     New York
1   Bob  Los Angeles
```

Output

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

    Name         City
0  Alice     New York
1    Bob  Los Angeles
```

Filter Rows Based on Conditions:

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}

df = pd.DataFrame(data)
print(df,"\n")
```

```
filtered = df.loc[df['Age'] > 25]
 # Select rows where Age > 25
print(filtered)
```

Output:

```
     Name        Age      City
1      Bob        30        Los Angeles
2  Charlie        35          Chicago
```

Output

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

      Name  Age         City
1      Bob   30  Los Angeles
2  Charlie   35      Chicago
```

Selecting Rows and Columns Using .iloc[] (Integer-Position Based Indexing)

The **.iloc[] method** selects **data based on integer positions (index numbers).**

It is particularly useful when you don't know the labels but know the positions.

Key Features of .iloc[]:

- Uses **integer positions (0, 1, 2, ...)** to index rows and columns.
- Just like .loc[], you can pass a range or a list of indices.
- **Supports slicing**, similar to Python lists.
- Unlike .loc[], it is exclusive when indexing ranges, meaning that the end index is excluded.

Select a Single Row by Position:

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}

df = pd.DataFrame(data)
print(df,"\n")

row = df.iloc[1]  # Select the second row (index position =
print(row)
```

Output:

```
Name          Bob
Age            30
City       Los Angeles
Name: 1, dtype: object
```

Output

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

Name            Bob
Age              30
City    Los Angeles
Name: 1, dtype: object
```

Select Multiple Rows by Positions:

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}

df = pd.DataFrame(data)
print(df,"\n")

rows = df.iloc[[0, 2]]  # Select first and third rows by p
print(rows)
```

Output:

```
      Name  Age      City
0    Alice   25  New York
2  Charlie   35   Chicago
```

Output

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

      Name  Age      City
0    Alice   25  New York
2  Charlie   35   Chicago
```

Select Specific Rows and Columns:

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}

df = pd.DataFrame(data)
print(df,"\n")

subset=df.iloc[[0,2],[1]]

print(subset)
```

Output:

```
   Age
0   25
2   35
```

Output

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

   Age
0   25
2   35
```

Differences Between loc and iloc

Feature	loc	iloc
Indexing Basis	Label-based (uses row/column labels)	Position-based (uses integer positions)
Inclusiveness	Inclusive of both start and end points	Exclusive of the end point (slicing)
Row Selection	Works with index labels (could be strings)	Works with integer positions (0, 1, 2, ...)
Column Selection	Works with column labels (can be strings)	Works with integer positions (0, 1, 2, ...)

3.3 Data Cleaning & Handling Missing Values (Functions)

https://www.geeksforgeeks.org/data-analysis/working-with-missing-data-in-pandas/

- **Identifying Missing Values: df.isnull(), df.isnull().sum().**

- isnull() returns a **DataFrame of Boolean** value where **True represents missing data (NaN).**

- isnull() identifies **NULL or NaN values** and returns a **boolean Series or DataFrame.**

- This is simple if we want to **find and fill missing data in a dataset.**
- **Syntax**:
  - **pd.isnull(obj)**
- **Parameters**:
  - **obj:** can be a Series, DataFrame, or scalar value.
  - **Returns:** Boolean Series/DataFrame with True for missing values.

```
import pandas as pd
import numpy as np

d = {'First Score': [100, 90, np.nan, 95],
     'Second Score': [30, 45, 56, np.nan],
     'Third Score': [np.nan, 40, 80, 98]}
df = pd.DataFrame(d)
print(df,"\n")

mv = df.isnull()

print(mv)
```

```
   First Score  Second Score  Third Score
0        100.0          30.0          NaN
1         90.0          45.0         40.0
2          NaN          56.0         80.0
3         95.0           NaN         98.0

   First Score  Second Score  Third Score
0        False         False         True
1        False         False        False
2         True         False        False
3        False          True        False
```

df.isnull().sum()

The df.isnull().sum() command is used in the **pandas** to count and display the total number of **missing or null values (NaNs)** in each column of a DataFrame (df).

```
import pandas as pd
import numpy as np

d = {'First Score': [100, 90, np.nan, 95],
     'Second Score': [30, 45, 56, np.nan],
     'Third Score': [np.nan, 40, 80, 98]}
df = pd.DataFrame(d)
print(df,"\n")

mv = df.isnull()

print(mv,"\n")

print(df.isnull().sum(),"\n")
df = df.fillna(0)
print(df,"\n")
```

```
   First Score  Second Score  Third Score
0        100.0          30.0          NaN
1         90.0          45.0         40.0
2          NaN          56.0         80.0
3         95.0           NaN         98.0

   First Score  Second Score  Third Score
0        False         False         True
1        False         False        False
2         True         False        False
3        False          True        False

First Score     1
Second Score    1
Third Score     1
dtype: int64

   First Score  Second Score  Third Score
0        100.0          30.0          0.0
1         90.0          45.0         40.0
2          0.0          56.0         80.0
3         95.0           0.0         98.0
```

https://www.geeksforgeeks.org/python/python-pandas-dataframe-dropna/

- **Techniques to Handle Missing Values: * Removing: df.dropna().**
- **Filling: df.fillna(value).**

- **DataFrame.dropna()** function remove missing values (NaN or None) from a DataFrame.
- It can **drop entire rows or columns depending** on the axis and threshold you specify.
- This method is commonly **used during data cleaning to eliminate incomplete data before analysis.**

- **Syntax**
- **DataFrame.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)**

- **Parameters:**

Parameter	Description
axis	0 to drop rows (default), 1 to drop columns
how	'any' (default): drop if any value is missing and 'all': drop if all are missing
thresh	Minimum number of non-NA values required to keep the row/column
subset	Labels to consider for NA checks (subset of columns)
inplace	If True, modifies the original DataFrame; if False (default), returns a new one

- **Returns:** A new DataFrame with the specified rows or columns removed unless **inplace=True**.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'A': [1, np.nan, 3], 'B': [4, 5, None]})

print(df, "\n")
print(df.dropna())
```

Output

```
     A    B
0  1.0  4.0
1  NaN  5.0
2  3.0  NaN

     A    B
0  1.0  4.0
```

Explanation:

By default, **dropna()** removes rows with any missing values.

Row 0 has no missing data, so it's kept.

Rows 1 and 2 contain NaN or None, so they're dropped. Only row 0 remains.

- **4. Selecting and Filtering Data**:
- This operation **retrieves specific columns, rows or records** that match a condition.
- It allows **precise extraction of required information.**

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie', 'Johny','Benny'],
    'Age': [25, 30, 35, 10,45],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Paris', 'California']}

df = pd.DataFrame(data)
print(df,"\n")

ages = df[df['Age'] > 25]
print(ages)
```

Output

```
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago
3    Johny   10        Paris
4    Benny   45   California

      Name  Age         City
1      Bob   30  Los Angeles
2  Charlie   35      Chicago
4    Benny   45   California
```

- **Explanation: df[df['age'] > 25]** returns rows where the "age" value is greater than 25.

- **5. Adding and Removing Columns**:
- You can create new columns based on existing ones or **delete unwanted columns** from the DataFrame.

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie', 'Johny','Benny'],
        'Age': [25, 30, 35, 10,45],
        'City': ['New York', 'Los Angeles', 'Chicago', 'Paris', 'California'],
    'Income':[5000,6000,3000,7000,1000],
    'Comm':[1500,4000,3000,2000,1500]
}

df = pd.DataFrame(data)
print(df,"\n")

df['Total_Salary'] = df['Income'] + df['Comm']

print(df)
```

Output

```
      Name  Age         City  Income  Comm
0    Alice   25     New York    5000  1500
1      Bob   30  Los Angeles    6000  4000
2  Charlie   35      Chicago    3000  3000
3    Johny   10        Paris    7000  2000
4    Benny   45   California    1000  1500

      Name  Age         City  Income  Comm  Total_Salary
0    Alice   25     New York    5000  1500          6500
1      Bob   30  Los Angeles    6000  4000         10000
2  Charlie   35      Chicago    3000  3000          6000
3    Johny   10        Paris    7000  2000          9000
4    Benny   45   California    1000  1500          2500
```

Explanation:
df['Total_Salary'] = df['Income'] + df['Comm']
creates a new column named "total".

- **6. Grouping Data (GroupBy)**:
- Grouping allows you **to organize data into categories** and compute values for each group
- for example, sums, counts or averages.

```
import pandas as pd

data = {'Name': ['Alice', 'Bob', 'Charlie', 'Johny','Benny'],
      'Age': [20, 30, 30, 10,20],
      'City': ['New York', 'Los Angeles', 'Chicago', 'Paris', 'California'],
   'Income':[5000,6000,3000,7000,1000],
   'Comm':[1500,4000,3000,2000,1500]
}

df = pd.DataFrame(data)
print(df,"\n")

res = df.groupby('Age')['Income'].sum()


print(res)
```

Output

```
      Name  Age         City  Income  Comm
0    Alice   20     New York    5000  1500
1      Bob   30  Los Angeles    6000  4000
2  Charlie   30      Chicago    3000  3000
3    Johny   10        Paris    7000  2000
4    Benny   20   California    1000  1500

Age
10    7000
20    6000
30    9000
Name: Income, dtype: int64
```

- **Explanation: df.groupby('Age')** divides the dataset based on the " **Age"** column and calculates its income

Empty Cells

https://www.w3schools.com/python/pandas/pandas_cleaning_empty_cells.asp

Empty cells can **potentially give you a wrong result when you analyze dat**a.

Remove Rows

- One way to deal with empty cells is to remove rows that contain empty cells.
- This is usually OK, since data **sets can be very big, and removing a few rows will not have a big impact on the result.**

- **Example :** Return a **new Data Frame with no empty cells:**

```
import pandas as pd

df = pd.read_csv('data.csv')

new_df = df.dropna()

print(new_df.to_string())
```

Note: By default, the **dropna() method returns a *new* DataFr**ame, and will not change the original.

- Duration,Date,Pulse,Maxpulse,Calories
- 60,'2020/12/01',110,130,409.1
- 60,'2020/12/02',117,145,479.0
- 60,'2020/12/03',103,135,340.0
- 45,'2020/12/04',109,175,282.4
- 45,'2020/12/05',117,148,406.0
- 60,'2020/12/06',102,127,300.0
- 60,'2020/12/07',110,136,374.0
- 450,'2020/12/08',104,134,253.3
- 30,'2020/12/09',109,133,195.1
- 60,'2020/12/10',98,124,269.0
- 60,'2020/12/11',103,147,329.3
- 60,'2020/12/12',100,120,250.7
- 60,'2020/12/12',100,120,250.7
- 60,'2020/12/13',106,128,345.3
- 60,'2020/12/14',104,132,379.3
- 60,'2020/12/15',98,123,275.0
- 60,'2020/12/16',98,120,215.2
- 60,'2020/12/17',100,120,300.0
- 45,'2020/12/18',90,112,
- 60,'2020/12/19',103,123,323.0
- 45,'2020/12/20',97,125,243.0
- 60,'2020/12/21',108,131,364.2
- 45,,100,119,282.0
- 60,'2020/12/23',130,101,300.0
- 45,'2020/12/24',105,132,246.0
- 60,'2020/12/25',102,126,334.5
- 60,20201226,100,120,250.0
- 60,'2020/12/27',92,118,241.0
- 60,'2020/12/28',103,132,
- 60,'2020/12/29',100,132,280.0
- 60,'2020/12/30',102,129,380.3
- 60,'2020/12/31',92,115,243.0

Python data.csv

```
Duration,Date,Pulse,Maxpulse,Calorie
60,'2020/12/01',110,130,409.1
60,'2020/12/02',117,145,479.0
60,'2020/12/03',103,135,340.0
45,'2020/12/04',109,175,282.4
45,'2020/12/05',117,148,406.0
60,'2020/12/06',102,127,300.0
60,'2020/12/07',110,136,374.0
450,'2020/12/08',104,134,253.3
30,'2020/12/09',109,133,195.1
60,'2020/12/10',98,124,269.0
60,'2020/12/11',103,147,329.3
60,'2020/12/12',100,120,250.7
60,'2020/12/12',100,120,250.7
60,'2020/12/13',106,128,345.3
60,'2020/12/14',104,132,379.3
60,'2020/12/15',98,123,275.0
60,'2020/12/16',98,120,215.2
60,'2020/12/17',100,120,300.0
45,'2020/12/18',90,112,
60,'2020/12/19',103,123,323.0
45,'2020/12/20',97,125,243.0
60,'2020/12/21',108,131,364.2
45,,100,119,282.0
60,'2020/12/23',130,101,300.0
45,'2020/12/24',105,132,246.0
60,'2020/12/25',102,126,334.5
60,20201226,100,120,250.0
60,'2020/12/27',92,118,241.0
60,'2020/12/28',103,132,
60,'2020/12/29',100,132,280.0
60,'2020/12/30',102,129,380.3
60,'2020/12/31',92,115,243.0
```

#Notice in the result that some rows have been removed **(row 18, 22 and 28).**

#These rows had cells with empty values.

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	2020/12/26	100	120	250.0
27	60	'2020/12/27'	92	118	241.0

- If you **want to change the original Da**taFrame,
  - use the inplace = True argument:

- **Example:-**Remove **all rows with NULL values:**

- import pandas as pd

  df = pd.read_csv('data.csv')

  df.dropna(inplace = True)

  print(df.to_string())

- **Note:** Now, the dropna(inplace = True) **will NOT return a new DataFra**me, but it will remove all rows containing NULL values from the original DataFrame.

#Notice in the result that **some rows have been removed (row 18, 22 and 28)**. #These rows had cells with empty values.

```
Duration,Date,Pulse,Maxpulse,Calories
60,'2020/12/01',110,130,409.1
60,'2020/12/02',117,145,479.0
60,'2020/12/03',103,135,340.0
45,'2020/12/04',109,175,282.4
45,'2020/12/05',117,148,406.0
60,'2020/12/06',102,127,300.0
60,'2020/12/07',110,136,374.0
450,'2020/12/08',104,134,253.3
30,'2020/12/09',109,133,195.1
60,'2020/12/10',98,124,269.0
60,'2020/12/11',103,147,329.3
60,'2020/12/12',100,120,250.7
60,'2020/12/12',100,120,250.7
60,'2020/12/13',106,128,345.3
60,'2020/12/14',104,132,379.3
60,'2020/12/15',98,123,275.0
60,'2020/12/16',98,120,215.2
60,'2020/12/17',100,120,300.0
45,'2020/12/18',90,112,
60,'2020/12/19',103,123,323.0
45,'2020/12/20',97,125,243.0
60,'2020/12/21',108,131,364.2
45,,100,119,282.0
60,'2020/12/23',130,101,300.0
45,'2020/12/24',105,132,246.0
60,'2020/12/25',102,126,334.5
60,20201226,100,120,250.0
60,'2020/12/27',92,118,241.0
60,'2020/12/28',103,132,
60,'2020/12/29',100,132,280.0
60,'2020/12/30',102,129,380.3
60,'2020/12/31',92,115,243.0
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5
26	60	2020/12/26	100	120	250.0
27	60	'2020/12/27'	92	118	241.0

Replace Empty Values

- Another way of **dealing with empty cells** is to insert a *new* value instead.
- This way you do not **have to delete entire rows just because of some empty cells.**
- The **fillna() method** allows us to replace empty cells with a value
- Example : Replace **NULL values with the number 130:**

```
import pandas as pd

df = pd.read_csv('data.csv')

df.fillna(130, inplace = True)
print(df)
```

#Notice in the result: **empty cells got the value 130 (in row 18, 22 and 28).**

```
Duration,Date,Pulse,Maxpulse,Calories
60,'2020/12/01',110,130,409.1
60,'2020/12/02',117,145,479.0
60,'2020/12/03',103,135,340.0
45,'2020/12/04',109,175,282.4
45,'2020/12/05',117,148,406.0
60,'2020/12/06',102,127,300.0
60,'2020/12/07',110,136,374.0
450,'2020/12/08',104,134,253.3
30,'2020/12/09',109,133,195.1
60,'2020/12/10',98,124,269.0
60,'2020/12/11',103,147,329.3
60,'2020/12/12',100,120,250.7
60,'2020/12/12',100,120,250.7
60,'2020/12/13',106,128,345.3
60,'2020/12/14',104,132,379.3
60,'2020/12/15',98,123,275.0
60,'2020/12/16',98,120,215.2
60,'2020/12/17',100,120,300.0
45,'2020/12/18',90,112,
60,'2020/12/19',103,123,323.0
45,'2020/12/20',97,125,243.0
60,'2020/12/21',108,131,364.2
45,,100,119,282.0
60,'2020/12/23',130,101,300.0
45,'2020/12/24',105,132,246.0
60,'2020/12/25',102,126,334.5
60,20201226,100,120,250.0
60,'2020/12/27',92,118,241.0
60,'2020/12/28',103,132,
60,'2020/12/29',100,132,280.0
60,'2020/12/30',102,129,380.3
60,'2020/12/31',92,115,243.0
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	130.0
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	130	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5

Replace Only For Specified Columns

- The example above **replaces all empty cells in the whole Data Frame.**

- To only replace empty values for one column, specify the ***column name*** **for the DataFrame.**

- **Replace NULL values** in the "**Calories**" columns with the number 130:

```
import pandas as pd

df = pd.read_csv('data.csv')

df.fillna({"Calories": 130}, inplace=True)
print(df)
```

#This operation inserts **130 in empty cells in the "Calories" column (row 18 and 28).**

```
Duration,Date,Pulse,Maxpulse,Calories
60,'2020/12/01',110,130,409.1
60,'2020/12/02',117,145,479.0
60,'2020/12/03',103,135,340.0
45,'2020/12/04',109,175,282.4
45,'2020/12/05',117,148,406.0
60,'2020/12/06',102,127,300.0
60,'2020/12/07',110,136,374.0
450,'2020/12/08',104,134,253.3
30,'2020/12/09',109,133,195.1
60,'2020/12/10',98,124,269.0
60,'2020/12/11',103,147,329.3
60,'2020/12/12',100,120,250.7
60,'2020/12/12',100,120,250.7
60,'2020/12/13',106,128,345.3
60,'2020/12/14',104,132,379.3
60,'2020/12/15',98,123,275.0
60,'2020/12/16',98,120,215.2
60,'2020/12/17',100,120,300.0
45,'2020/12/18',90,112,
60,'2020/12/19',103,123,323.0
45,'2020/12/20',97,125,243.0
60,'2020/12/21',108,131,364.2
45,,100,119,282.0
60,'2020/12/23',130,101,300.0
45,'2020/12/24',105,132,246.0
60,'2020/12/25',102,126,334.5
60,20201226,100,120,250.0
60,'2020/12/27',92,118,241.0
60,'2020/12/28',103,132,
60,'2020/12/29',100,132,280.0
60,'2020/12/30',102,129,380.3
60,'2020/12/31',92,115,243.0
```

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
2	60	'2020/12/03'	103	135	340.0
3	45	'2020/12/04'	109	175	282.4
4	45	'2020/12/05'	117	148	406.0
5	60	'2020/12/06'	102	127	300.0
6	60	'2020/12/07'	110	136	374.0
7	450	'2020/12/08'	104	134	253.3
8	30	'2020/12/09'	109	133	195.1
9	60	'2020/12/10'	98	124	269.0
10	60	'2020/12/11'	103	147	329.3
11	60	'2020/12/12'	100	120	250.7
12	60	'2020/12/12'	100	120	250.7
13	60	'2020/12/13'	106	128	345.3
14	60	'2020/12/14'	104	132	379.3
15	60	'2020/12/15'	98	123	275.0
16	60	'2020/12/16'	98	120	215.2
17	60	'2020/12/17'	100	120	300.0
18	45	'2020/12/18'	90	112	130.0
19	60	'2020/12/19'	103	123	323.0
20	45	'2020/12/20'	97	125	243.0
21	60	'2020/12/21'	108	131	364.2
22	45	NaN	100	119	282.0
23	60	'2020/12/23'	130	101	300.0
24	45	'2020/12/24'	105	132	246.0
25	60	'2020/12/25'	102	126	334.5

dropna()

- The dropna() method removes the rows that contains NULL values.

Syntax :

- *dataframe*.dropna(axis, how, thresh, subset, inplace)

Parameters:

- The axis, how, thresh, subset, inplace, parameters are keyword arguments.

Return Value:

- A DataFrame with the result, or None if the inplace parameter is set to True.

Parameter	Value	Description

axis	`0`
	`1`
	`'index'`
	`'columns'`
		Optional, default 0.
		0 and 'index'removes ROWS that contains NULL values
		1 and 'columns' removes COLUMNS that contains NULL values

how	`'all'` `'any'`	Optional, default 'any'. Specifies whether to remove the row or column when ALL values are NULL, or if ANY value is NULL.
thresh	*Number*	Optional, Specifies the number of NOT NULL values required to keep the row.
subset	*List*	Optional, specifies where to look for NULL values
inplace	`True` `False`	Optional, default False. If True: the removing is done on the current DataFrame. If False: returns a copy where the removing is done.

fillna()

- The fillna() method replaces the NULL values with a specified value.

- The fillna() method returns a new DataFrame object unless the inplace parameter is set to True, in that case the fillna() method does the replacing in the original DataFrame instead

- **Syntax:**

- ***dataframe*.fillna(value, method, axis, inplace, limit, downcast)**

Parameters

The axis, method, inplace, limit, downcast parameters are keyword arguments.

Parameter	Value	Description
value	*Number* *String* *Dictionary* *Series* *DataFrame*	Required, Specifies the value to replace the NULL values with. This can also be values for the entire row or column.

method	`'backfill'`
	`'bfill'`
	`'pad'`
	`'ffill'`
	`None`
		Optional, default None'. Specifies the method to use when replacing

axis	`0`
	`1`
	`'index'`
	`'columns'`
		Optional, default 0. The axis to fill the NULL values along

inplace	`True` `False`	Optional, default False. If True: the replacing is done on the current DataFrame. If False: returns a copy where the replacing is done.
limit	*Number* `None`	Optional, default None. Specifies the maximum number of NULL values to fill (if method is specified)
downcast	*Dictionary* `None`	Optional, a dictionary of values to fill for specific data types

Removing Duplicate and Inconsistent Data: df.duplicated()

- **duplicated() Method**
- The **duplicated()** method helps to **identify duplicate rows** in a dataset.

```
import pandas as pd
data = {
    'Name': ['Alice', 'Bob', 'Alice', 'Charlie','Bob', 'David'],
    'Age': [25, 30, 25, 35, 30, 40],
    'City': ['New York', 'Los Angeles', 'New York','Chicago', 'Los Angeles', 'San Francisco']
}

df = pd.DataFrame(data)
print(df,"\n")

duplicates = df.duplicated()
print(duplicates)
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Alice	25	New York
3	Charlie	35	Chicago
4	Bob	30	Los Angeles
5	David	40	San Francisco

It returns a boolean Series indicating whether a row is a duplicate of a previous row.

- df.duplicated() identifies rows where **every column value** is an identical match to a previous row [1].

- **How it works:**
- **First Occurrence:** The first time a unique row appears, it is marked False (not a duplicate) [1, 2].

- **Subsequent Occurrences:** If the same data appears again later in the DataFrame, it is marked True [1, 3].

Output

```
      Name  Age           City
0    Alice   25       New York
1      Bob   30    Los Angeles
2    Alice   25       New York
3  Charlie   35        Chicago
4      Bob   30    Los Angeles
5    David   40  San Francisco

0    False
1    False
2     True
3    False
4     True
5    False
dtype: bool
```

```
import pandas as pd
data = {
    'Name': ['Alice', 'Bob', 'Alice', 'Charlie',
                                'Bob', 'David'],
    'Age': [25, 30, 25, 35, 30, 40],
    'City': ['New York', 'Los Angeles', 'New York',
                 'Chicago', 'Los Angeles', 'San Francisco']
}

df = pd.DataFrame(data)
print(df,"\n")

duplicates = df[df.duplicated()]
print(duplicates)
```

```
      Name  Age           City
0    Alice   25       New York
1      Bob   30    Los Angeles
2    Alice   25       New York
3  Charlie   35        Chicago
4      Bob   30    Los Angeles
5    David   40  San Francisco

    Name  Age         City
2  Alice   25     New York
4    Bob   30  Los Angeles
```

- **How it Works**
- The line duplicates = df[df.duplicated()] uses **boolean indexing** to filter the data:

- **df.duplicated()**: This method scans the DataFrame row by row. By default, it returns a Series of True and False values. A row is marked True only if it is an **exact match** (all column values are the same) of a row that appeared earlier in the list.

- **df[...]**: The DataFrame uses those True/False values as a filter. It ignores everything marked False (the unique and first-occurrence rows) and keeps only the rows marked True.

```
      Name  Age           City
0    Alice   25       New York
1      Bob   30    Los Angeles
2    Alice   25       New York
3  Charlie   35        Chicago
4      Bob   30    Los Angeles
5    David   40  San Francisco

    Name  Age         City
2  Alice   25     New York
4    Bob   30  Los Angeles
```

df.drop_duplicates(): Remove duplicates.

- The drop_duplicates() method **removes duplicate rows.**

- Use the **subset parameter** if only some specified columns should be considered when **looking for duplicates.**

Syntax:

- ***dataframe*.drop_duplicates(subset, keep, inplace, ignore_index)**

Parameters:

- The parameters are keyword arguments.

Parameter	Value	Description
subset	column label(s)	Optional. A String, or a list, containing the columns to use when looking for duplicates. If not specified, all columns are being used.

keep	`'first'`
	`'last'`
	`False`
		Optional, default 'first'. Specifies which duplicate to keep. If False, drop ALL duplicates

inplace	`True` `False`	Optional, default False. If True: the removing is done on the current DataFrame. If False: returns a copy where the removing is done.
ignore_index	`True` `False`	Optional, default False. Specifies whether to label the 0, 1, 2 etc., or not

- **Return Value**

A DataFrame with the result, or None if the inplace parameter is set to True.

```
import pandas as pd

data = {
  "name": ["Sally", "Mary", "John", "Mary"],
  "age": [50, 40, 30, 40],
  "qualified": [True, False, False, False]
}

df = pd.DataFrame(data)
print(df,"\n")

newdf = df.drop_duplicates()
print(newdf)
```

Output

```
    name  age  qualified
0  Sally   50       True
1   Mary   40      False
2   John   30      False
3   Mary   40      False

    name  age  qualified
0  Sally   50       True
1   Mary   40      False
2   John   30      False
```

Pandas dataframe.replace()

- The replace() method replaces the specified value with another specified value.

- The replace() method searches **the entire DataFrame** and replaces every case of the **specified value**.

Syntax :

- ***dataframe*.replace(*to_replace*, *value*, inplace, limit, regex, method)**

Parameters :

- The inplace, limit, regex, method parameters are **keyword arguments.**

Return Value :

- A DataFrame with the result, or None if the inplace parameter is set to **True**.

Parameter	Value	Description
to_replace		Required, a String, List, Dictionary, Series, Number, or a Regular Expression describing what to search for
value		Optional, A String, Number, Dictionary, List or Regular Expression that specifies a value to replace with.
inplace	`True` `False`	Optional, default False. If True: the replacing is done on the current DataFrame. If False: returns a copy where the replacing is done.
limit	Number `None`	Optional, default None. Specifies the max limit of the size gap to fill

regex	`True`
	`False`
	`None`
		Optional, default False. Specifies whether to replace using a regular expression or not.

method	`'bfill'`
	`'ffill'`
	`'pad'`
	`None`
		Optional, Specifies the how to replace if the `value` parameter is not present.

```
import pandas as pd

data = {
  "name": ["Bill", "Bob", "Betty"],
  "age": [50, 50, 30],
  "qualified": [True, False, False]
}

df = pd.DataFrame(data)
print (df)

newdf = df.replace(50, 60)
print (newdf)
```

Output

```
    name  age  qualified
0   Bill   50       True
1    Bob   50      False
2  Betty   30      False

    name  age  qualified
0   Bill   60       True
1    Bob   60      False
2  Betty   30      False
```

Data Imputation Techniques in ML

- Addressing missing data is a critical pre-processing step and this is where data imputation techniques come into play.

- Missing data arises due to various reasons such as human errors, device **malfunctions or software issues.**

Missing data can be classified into three categories:

- **Missing Completely at Random (MCAR)**
- **Missing at Random (MAR)**
- **Missing Not at Random (MNAR)**

Data Imputation Techniques in ML

- **Missing Completely at Random (MCAR):** The probability of a value being missing is independent of the variable itself or any other variables.

- The missingness is pure chance, such as a random technical glitch, a lost survey, or a machine malfunctioning at random times.

- **Examples of MCAR:**
- **Lost Survey Pages:** A researcher gathers 100 survey responses, but 5 are lost in the mail. The missing answers (e.g., income, age) are missing because the paper was lost, not because of what the participants wrote.

- **Malfunctioning Sensor:** A temperature sensor in a factory breaks down randomly for 10 minutes every day. The missing temperature data is independent of the actual temperature or time of day.

- In these scenarios, the missing cases do not differ systematically from the observed cases, making the data set an unbiased sample of the whole.

Data Imputation Techniques in ML

- **Missing at Random (MAR):** The probability of missingness is related to other observed variables but not the missing variable itself.

- Missing at Random (MAR) occurs when the probability of data being missing is related to observed data (such as age, gender, or location) but not the actual missing values themselves.

- The missing information can be predicted by other variables in the dataset.
- **Examples of MAR:**
- **Survey Data:** Income data is missing, but it **is missing more often for younger people (observed variable) than older people. If** we know the age, we can account for the missingness.

- **Medical Studies:** Male participants are less likely to **complete a depression survey than females, but the missingness is not related to their actual level of depression, only to their gender.**

- MAR data can often be handled using imputation methods, as the reason for the missingness can be modeled using the observed data.

Data Imputation Techniques in ML

- **Missing Not at Random (MNAR):** The probability of **missingness depends on the variable that is missing.**

- Missing Not at Random (MNAR) occurs when the probability of data being missing is directly related to the missing values themselves, causing systematic bias.

- For example, in a health survey, people with severe depression may be less likely to report their mood, resulting in missing data for the lowest values.

- **Examples:**
- **Income Surveys:** High-income earners are less likely to disclose their income, causing high-income values to be disproportionately missing.

- **Performance Metrics:** An employee with low sales performance is less likely to report their numbers in a self-assessment survey.

- MNAR is challenging because the cause of missingness is unobserved, often requiring complex, specialized modelling to fix.

Importance of Data Imputation

- **Model Training:** Missing data reduces the amount of usable data, affecting the **accuracy and training** of the model.

- **Bias Avoidance:** Proper imputation reduces the risk of biased inferences.

- **Data Integrity:** Imputation ensures **completeness**, leading to robust analysis and predictions.

Imputation Techniques

1. Mean/Median Imputation

The missing values are replaced by the mean (or median) of the non-missing values.

$$\text{Imputed Value} = \frac{\sum_{i=1}^{n} X_i}{n}, \text{ where } X_i \neq \text{NaN}.$$

Handling Missing Values Using Mean Imputation in NumPy:

```
import numpy as np
#Create a dataset with a missing value
data = np.array([1, 2, np.nan, 4, 5])

print(data,"\n")
# Compute the mean excluding NaN values
mean_value = np.nanmean(data)

print("Mean value is ",mean_value,"\n")

#Replace NaN with the computed mean
data = np.where(np.isnan(data), mean_value, data)
print(data)
```

Output

```
[ 1.  2. nan  4.  5.]

Mean value is  3.0

[1. 2. 3. 4. 5.]

=== Code Execution Successful ===
```

2. Mode Imputation

- Replace **missing values with the mode** (most frequent value).

- This is especially useful for categorical data.

- **Handling Missing Categorical Data Using Mode Imputation in Python:**

```
import numpy as np
from collections import Counter
data = np.array(['cat', 'dog', None, 'cat'], dtype=object)   # Use None instead of np.nan
print(data,"\n")

filtered_data = [x for x in data if x is not None]  # Remove None values
mode_value = Counter(filtered_data).most_common(1)[0][0]                    # Find most frequent category
print("Mode value = ", mode_value,"\n")

# Replace None with the computed mode
data = np.array([x if x is not None else mode_value for x in data], dtype=object)
print(data)
```

Output

```
['cat' 'dog' None 'cat']

Mode value =  cat

['cat' 'dog' 'cat' 'cat']
```

- **Step-by-Step Breakdown**
- **Data Preparation**:

  It creates an array with categorical data (cat, dog) and a missing value (None).

  Because it contains mixed types, it is explicitly set to dtype=object.
- **Filtering**:

  filtered_data = [x for x in data if x is not None]

  Before calculating the mode, it creates a temporary list that excludes

  the None value. You can't calculate a "most frequent" category if the missing data

  itself is included in the count.

- **Finding the Mode**:

  mode_value = Counter(filtered_data).most_common(1)[0][0]

  - **Counter(filtered_data)**: Creates a dictionary-like object that counts occurrences (e.g., {'cat': 2, 'dog': 1}).
  - **.most_common(1)**: Returns a list of the top 1 frequent items as a tuple: [('cat', 2)].
  - **[0][0]**: Drills down into that list and tuple to extract just the string 'cat'.

- **Imputation (Filling the Gap)**:
- data = np.array([x if x is not None else mode_value for x in data], dtype=object)
  This uses a list comprehension to rebuild the array.
- It checks every item: if it’s a valid value, it stays; if it’s None, it gets replaced by the mode_value ('cat').

- **Final Result**
- The original array ['cat' 'dog' None 'cat'] becomes ['cat' 'dog' 'cat' 'cat'].

```
import numpy as np
from collections import Counter
data = np.array(['5', '4', None, '2' , '4', '3'], dtype=object)   # Use None instead of np.nan
print(data,"\n")

filtered_data = [x for x in data if x is not None]  # Remove None values
mode_value = Counter(filtered_data).most_common(1)[0][0]  # Find most frequent
    category
print("Mode value = ", mode_value,"\n")

# Replace None with the computed mode
data = np.array([x if x is not None else mode_value for x in data], dtype=object)
print(data)
```

Output

```
['5' '4' None '2' '4' '3']

Mode value =  4

['5' '4' '4' '2' '4' '3']
```

Thank you